



SISTEMAS OPERATIVOS - TEORÍA

8 de Enero de 2025

Nombre _____ DNI _____
Apellidos _____ Grupo _____

Cuestión 1 (2 puntos). Se dispone de un **sistema de archivos** tipo UNIX muy **simplificado** para un entorno embebido. Este sistema usa bloques de **32 bytes** y **direcciones de bloque de 1 byte**. Cada nodo-i cuenta con un **único enlace directo** y un **único enlace indirecto simple**.

Máscara de bits: 1 1 1 1 1 0 1 0 1 1 1...

Tabla de nodos-i

nodo-i	2	3	4	5	6	7	8
Tam	2	1	1	?	1	?	?
#enlaces	3	2	2	?	1	1	?
Tipo F/D	D	D	D	D	F	F	F
Permisos	rwX	r-x	rwX	rwX	rw-	rw-	rw-
Directo	0	4	1	9	2	6	3
Indirecto	Null	Null	Null	Null	Null	8	Null

Bloques:

0		1		2	3	4		6	8	9		10
.	2	.	4	AAAAAAAA	CCCCCCCC	.	3	BBBBBBBB	10	.	5	DDDDDDDD
..	2	..	2	AAAAAAAA	CCCCCCCC	..	2	BBBBBBBB		..	3	DDDDDDDD
user	3	A	6	AAAAAAAA	CCCCCCCC	extra	5	BBBBBBBB				DDDDDDDD
note	4	B	7	AAAAAAAA	CCCCCCCC			BBBBBBBB				DDDDDDDD
s												DDDDDDDD

```
int main(void) {  
    char buf[32] = {[0 ... 31] = 'X'};  
    int fd, n;  
    fd = open("/notes/A", O_RDWR);  
    lseek(fd, 32, SEEK_SET);  
    n = write(fd, buf, 32);  
    lseek(fd, -16, SEEK_CUR);  
    n = read(fd, buf, 16);  
    close(fd);  
    return 0;  
}
```

a) (0.75 puntos) Dibuja el árbol de directorios a partir de la información dada, completa las entradas que faltan en la tabla de i-nodos.

b) (0.75 puntos) Después de ejecutar la operación **write()** sobre el fichero **/notes/A**, explica detalladamente cómo queda el i-nodo del fichero A. Indica si esta instrucción ha provocado la creación de un bloque indirecto y la asignación de nuevos bloques de datos. Describe el estado final del i-nodo asociado al fichero A y, en caso de que se haya creado un nuevo bloque indirecto, cómo queda este bloque y los datos que contiene. Actualiza también el Mapa de Bits en consecuencia.

c) (0.5 puntos) Una vez que hemos ejecutado esas operaciones anteriores, explica qué hace el comando **ln -s /notes/A /user/extra** si lo hacemos ahora mismo. ¿Crea un nuevo i-nodo? ¿Crea un nuevo bloque? Muestra la nueva entrada creada en el directorio **/user/extra** y el nuevo i-nodo del enlace simbólico (si lo hubiere), indicando el contenido de su bloque de datos con la ruta destino. Actualiza la tabla de i-nodos con este nuevo i-nodo.

Cuestión 2 (2 puntos). El siguiente programa examina los rangos de direcciones virtuales válidos de su mapa de memoria virtual y el de un proceso hijo creado. Utiliza para ello la llamada al sistema `system`, que crea un proceso y hace que ejecute un shell que interprete el comando que se pasa como argumento a `system`, y espera a que dicho proceso termine. En el programa el comando que se pasa vuelca por la salida estándar el contenido del fichero `maps` del proceso cuyo `pid` se pasa a `print_map`, filtrando alguna de las columnas y filas que muestra. Para cada región se muestra su rango de direcciones virtuales, su protección y la ruta del fichero de soporte en caso de tenerlo (también identifica algunas regiones sin soporte como el `heap`).

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <pthread.h>
#define BUFSIZE 1024
char buf[BUFSIZE];

int print_map(pid_t pid) {
    int fd;
    char b;
    printf("Map of process %d:\n", (int) pid);
    printf("<inicio>--<fin>      <modo>\t\t      <fichero de soporte>\n");
    snprintf(buf, BUFSIZE, "cat /proc/%d/maps |grep -v 'vvar\\|vdso' |cut -d' ' --complement -f3,4,5",
        (int) pid);
    system(buf);
    return 0;
}

void *entry(void *arg) {
    int a = 5;
    printf("a: %p\n", &a);
    print_map(getpid());
    return NULL;
}

int main(int argc, char **argv) {
    pid_t pid;
    pthread_t thid;
    int *p = mmap(NULL, 4096, PROT_READ | PROT_WRITE, MAP_SHARED | MAP_ANONYMOUS, 0, 0);
    printf("pid: %p\n", &pid);
    printf("buf: %p\n", buf);
    printf("main: %p\n", main);
    printf("fork: %p\n", fork);
    printf("format string: %p\n", "fork: %p\n");
    pid = fork();
    if (pid == 0) {
        pthread_create(&thid, NULL, entry, NULL);
        pthread_join(thid, NULL);
        exit(EXIT_SUCCESS);
    } else {
        wait(NULL);
        print_map(pid);
        print_map(getpid());
    }
    return 0;
}
```

Al ejecutar el programa **hasta la primera invocación de la función `print_map` incluida** la salida estándar es:

```
$ ./memoria
pid: 0x7ffc1425d1f4
buf: 0x4bd320
main: 0x401757
fork: 0x43b690
format string: 0x48d0cb
a: 0x7f90797ce24c
```

```

Map of process 34660:
<inicio>-<fin>      <modo>                                <archivo de soporte>
00400000-00401000  r--p                                     /home/usuario/memoria
00401000-0048d000  r-xp                                     /home/usuario/memoria
0048d000-004b7000  r--p                                     /home/usuario/memoria
004b7000-004bb000  r--p                                     /home/usuario/memoria
004bb000-004be000  rw-p                                     /home/usuario/memoria
004be000-004c4000  rw-p
0092c000-0094e000  rw-p                                     [heap]
7f9078fce000-7f9078fcf000  ---p
7f9078fcf000-7f90797cf000  rw-p
7f90797cf000-7f90797d0000  rw-s                             /dev/zero (deleted)
7ffc1423e000-7ffc1425f000  rw-p                             [stack]

```

- a. Para cada uno de los rangos del mapa de memoria del proceso 34660 mostrados, razona qué región (o tipo de región) es, indicando si existe desde la creación del proceso o si ha sido creada dinámicamente en algún momento, señalando en tal caso la línea de código responsable de su creación. ¿El programa está enlazado estática o dinámicamente? Justifica tu respuesta.

- b. Describe la salida estándar que se observará tras la invocación de las dos últimas invocaciones de `print_map`.

Cuestión 3 (2 puntos). En un sistema GNU/Linux, se ejecuta el siguiente programa desde el home de un usuario:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#define N 3

int main(int argc, char **argv) {
    pid_t pid;
    int i,status;
    char buf[100];
    int fd1,fd2;

    printf("Original process %d\n", (int) getpid());
    fd1 = open("log1.txt", O_WRONLY | O_CREAT | O_TRUNC, 0600);
    fd2 = open("log2.txt", O_WRONLY | O_CREAT | O_TRUNC, 0600);
    close(fd2);
    for (i = 0; i < N; i++) {
        pid = fork();
        if (pid == 0) {
            fd2 = open("log2.txt", O_WRONLY);
            snprintf(buf, 100, "hello from %d with i=%d\n", (int) getpid(), i);
            if ((i % 2) == 0) {
                execl("echo", "echo", "-n", buf, NULL);
            } else {
                execlp("echo", "echo", "-n", buf, NULL);
            }
            write(fd1, buf, strlen(buf));
            write(fd2, buf, strlen(buf));
            close(fd2);
        } else if (pid > 0) {
            wait(&status);
            if (WIFEXITED(status)) {
                printf("My pid is %d, my child %d exit value was %d\n",
                    (int) getpid(),
                    (int) pid,
                    WEXITSTATUS(status));
            }
        }
    }
    close(fd1);
    return 0;
}
```

Sabiendo que la variable de entorno PATH contiene las rutas de búsqueda habituales, responde a las siguientes preguntas:

- a. Indica cuántos procesos se crearán en total (sin contar con el proceso original) y cuántos de ellos podrán ejecutar concurrentemente, justificando la respuesta.

- b. Dibuja el grafo de creación de los procesos y razona si es posible o no determinar el orden exacto en el que se ejecutarán estos procesos.

- c. Describe lo que podrá verse por la salida estándar del proceso al ejecutar el programa asumiendo que el sistema asigna pids consecutivos a todos los procesos en el orden en que son creados (por ejemplo empezando por el pid 100 para el proceso original).

- d. Describe también el contenido final de los ficheros log1.txt y log2.txt tras la ejecución completa del programa,

Cuestión 4. (2 puntos) En una vuelta ciclista por etapas, todos los ciclistas deben completar una serie de etapas asegurándose de que nadie inicie una nueva etapa hasta que el último ciclista haya finalizado la etapa actual. Este mecanismo garantiza que todos los ciclistas comiencen cada etapa al mismo tiempo. Se debe implementar un programa que gestione esta sincronización utilizando hilos, mutex y variables condicionales. El escenario es el siguiente:

1. Hay N ciclistas (representados como hilos) y cada uno debe completar M etapas.
2. La función `correr_etapa(id, etapa)` (ya implementada) simula el tiempo que un ciclista tarda en completar una etapa. Cada ciclista tarda un tiempo aleatorio en finalizar la etapa.
3. La función `esperar_fin_de_etapa(etapa)` debe ser implementada. Esta función es bloqueante y debe garantizar que:
 - Un ciclista se quede esperando al final de la etapa hasta que todos los ciclistas hayan completado la misma etapa.
 - Solo cuando todos los ciclistas hayan llegado al final de la etapa actual, podrán comenzar simultáneamente la siguiente etapa.

```
// Constantes y variables globales
```

```
// ...
```

```
void* ciclista(void* arg) {  
    int id = *(int*)arg;  
    for (int etapa = 0; etapa < M; etapa++) {  
        correr_etapa(id, etapa);  
        esperar_fin_de_etapa(etapa);  
    }  
    return NULL;  
}
```

Cuestión 5 (2 puntos): Se dispone del siguiente código de un módulo de kernel (*kernmod.c*), que implementa un comportamiento similar a */dev/null*, con la capacidad añadida de recordar cuántos caracteres se han volcado.

```
//includes, defines, fops y otras operaciones se asumen correctamente inicializadas
static int letter_count = 0;

static int device_open(struct inode *inode, struct file *file) {
    printk(KERN_INFO "kernmod: Device opened\n");
    return 0;
}

static int device_release(struct inode *inode, struct file *file) {
    printk(KERN_INFO "kernmod: Device closed\n");
    return 0;
}

static ssize_t device_read(struct file *file, char __user *buf, size_t len, loff_t *off) {
    char message[32];
    int message_len = scnprintf(message, sizeof(message), "Recibido: %d\n", letter_count);
    int return_len;
    if (*off >= message_len) // Indicar EOF devolviendo 0
        return 0;
    message_len -= *off;
    return_len = (len < message_len) ? len : message_len;
    if (copy_to_user(buffer, message + *off, return_len))
        return -EFAULT;
    *off += return_len;    // Actualizar offset
    return return_len;
}

static ssize_t device_write(struct file *file, const char __user *buf, size_t len, loff_t *off) {
    letter_count += len;
    printk(KERN_INFO "kernmod: %zu letras recibidas, total: %d\n", len, letter_count);
    return len;
}

int init_module (void) {
    //Código de la función init_module. Llama a las funciones
    //alloc_chrdev_region, cdev_alloc y cdev_add, inicializando correctamente el módulo.
    //Además, indica a través de printk que se asocian Major=251 y Minor=0
    ...
}

void cleanup_module(void) {
    //Código de la función cleanup_module. Elimina correctamente el dispositivo
    //mediante cdev_del y unregister_chrdev_region.
    //Indica mediante printk que el módulo se ha descargado
    ...
}
```

El módulo se compila como *kernmod.ko*. Se ejecutan las siguientes instrucciones en la shell:

```
1> sudo insmod kernmod.ko
2> mknod -m 666 /dev/kernmod c 251 0
3> echo "Hello" > /dev/kernmod
4> cat /dev/kernmod
5> sudo rmmod kernmod.ko
```

Responda a las siguientes preguntas, razonando su respuesta:

- a. ¿Qué diferencia obtenemos si cambiamos *printk* por la equivalente *printf*?

b. ¿Cuál/es de las 6 funciones del módulo del kernel son invocadas con cada una de las 5 órdenes anteriores?

c. ¿Qué permisos adquiere el fichero **/dev/kernmod**? ¿Qué tipo de fichero es? ¿Cuánto ocupa en disco antes y después de la orden 3?

d. Modifica **device_write** para que devuelva **-EFAULT** si una dirección del rango **(buf, buf+len)** es *inválida*.

e. Explica qué sucede si, tras la orden 5, ejecutamos de nuevo el comando **cat /dev/kernmod**.